Technologies du projet PSG Académie

---

1. BACKEND - Framework & Serveur

FastAPI 0.124.4

- Rôle : Framework web Python pour créer des API REST
- Avantages :
  - ⚡ Performance : Plus rapide que Django/Flask (basé sur Starlette + Pydantic)
  - 📝 Documentation auto : Génère OpenAPI/Swagger automatiquement (/docs)
  - ✅ Validation : Vérifie automatiquement les données entrantes avec Pydantic
  - 🔄 Async natif : Support async/await pour hautes performances
  - 💡 Autocomplétion : Typage fort → meilleur support IDE
- Alternatives : Django REST (plus lourd), Flask (moins moderne), Express.js (Node.js)

Uvicorn 0.34.0

- Rôle : Serveur ASGI pour exécuter FastAPI
- Avantages :
  - ⚡ Ultra-rapide (basé sur uvloop)
  - 🔁 Hot reload en développement
  - 🔄 Support async natif
  - 🚀 Production-ready avec workers multiples
- Alternatives : Gunicorn (WSGI, moins performant), Hypercorn

Nginx

- Rôle : Serveur web de production et reverse proxy
- Avantages :
  - 🚀 Reverse proxy : Dirige le trafic vers FastAPI/Uvicorn
  - 🔒 HTTPS facile avec Let's Encrypt (certbot)
  - 📦 Fichiers statiques : Sert React build ultra-rapidement
  - ⚖️ Load balancing : Répartit charge sur plusieurs instances
  - 🛡️ Sécurité : Rate limiting, protection DDoS, headers de sécurité

    - 📊 Gestion compression gzip/brotli
- Alternatives : Apache (plus lourd, configuration complexe), Caddy (auto HTTPS mais moins configurable), Traefik

---

2. BACKEND - Base de données

PostgreSQL 16

- Rôle : Base de données relationnelle principale
- Avantages :
    - 🏆 Open source leader en fonctionnalités avancées
    - 🔗 Relations complexes : Foreign keys, joins performants (Country→Academy→Team→Player)
    - 📊 Types riches : JSON, arrays, dates avancées, géométrie
    - 🔒 ACID compliance : Transactions fiables et cohérentes
    - 🚀 Excellent pour analytics : Window functions, CTEs, indexation avancée
    - 🔧 Extensions puissantes (PostGIS, pgcrypto, etc.)
- Alternatives : MySQL (moins de features avancées), SQLite (fichier simple, mono-utilisateur), MongoDB (NoSQL, pas de relations)

SQLModel 0.0.27 *(backend-catapult)*

- Rôle : ORM pour le backend Catapult
- Avantages :
    - 🎯 Best of both worlds : Combine SQLAlchemy (ORM puissant) + Pydantic (validation)
    - ✨ Moins de code : Une seule classe = modèle DB + schéma API
    - 🔍 Type safety : Détection d'erreurs avant exécution
    - 📖 Créé par le même auteur que FastAPI (intégration parfaite)
    - 🔄 Migrations faciles avec Alembic
- Alternatives : SQLAlchemy pur (plus verbeux), Django ORM (lié à Django)

SQLAlchemy 2.0.25 *(backend-veo)*

- Rôle : ORM pour le backend Veo
- Avantages :
    - 🏆 Standard de facto Python ORM, très mature

    - 🔧 Très flexible : Requêtes complexes, relations avancées
    - 🔄 Compatible Alembic pour les migrations
    - 📖 Documentation exhaustive, large communauté
- Alternatives : SQLModel (plus récent), Django ORM (lié à Django)

Alembic 1.14.0

- Rôle : Migrations de base de données
- Avantages :
    - 🔄 Versionning du schéma SQL
    - 🔧 Génération automatique des migrations
    - 🚀 Compatible SQLAlchemy et SQLModel
    - 💾 Rollback possible en cas d'erreur
- Alternatives : Django migrations (lié à Django), Flyway (Java)

psycopg2-binary 2.9.10

- Rôle : Driver PostgreSQL pour Python
- Avantages :
    - ⏳ Standard de facto pour PostgreSQL en Python
    - ⚡ Version binary = pré-compilée (pas besoin de compiler libpq)
    - 🔒 Très stable et mature
    - 🔧 Pool de connexions intégré
- Alternatives : asyncpg (async pur, plus complexe), psycopg3 (nouvelle version)

---

3. BACKEND - Traitement de données

Pandas 2.3.3

- Rôle : Analyse et manipulation de données tabulaires
- Avantages :
    - 📊 Standard industrie pour data science en Python
    - 🔧 Parsing CSV puissant : Détection auto de types, gestion encodages
    - 🎯 Groupby/Aggregate : Grouper par joueur, calculer moyennes/max
    - 🔄 Transformations : Pivot, merge, join de dataframes
    - 📈 Compatible Matplotlib/Seaborn pour visualisations

- Mon usage : Parser CSV Catapult GPS, grouper par joueur, calculer benchmarks
- Alternatives : Polars (plus rapide mais moins mature), CSV stdlib (trop basique)

Matplotlib 3.10.8

- Rôle : Génération de graphiques et visualisations côté serveur
- Avantages :
  - 🎨 Contrôle total : Personnalisation pixel par pixel
  - 📊 Tous types de graphiques : Gauges semi-circulaires custom, heat maps, etc.
  - 💾 Export : PNG/PDF/SVG haute qualité
  - 🏆 Bibliothèque la plus utilisée en Python (écosystème énorme)
  - 🔧 Backend Agg pour génération server-side sans display
- Mon usage : Rapports professionnels avec header, logo, jauges, tableaux colorés
- Alternatives : Plotly (interactif HTML mais lourd), Bokeh (web-based)

Seaborn 0.13.2

- Rôle : Visualisations statistiques élégantes
- Avantages :
  - 🎨 Thèmes pro par défaut (vs Matplotlib brut)
  - 📊 Graphiques statistiques en 1 ligne (distributions, corrélations)
  - 🔧 S'intègre parfaitement avec Matplotlib (même backend)
  - 🎨 Palettes de couleurs harmonieuses
- Mon usage : Color schemes, esthétique professionnelle des rapports
- Alternatives : Matplotlib pur (moins joli), Plotly Express

Pillow 12.0.0

- Rôle : Manipulation d'images (fork moderne de PIL)
- Avantages :
  - 🖼️ Ouvre/redimensionne/affiche images
  - 🔧 Intégration Matplotlib : OffsetImage, AnnotationBbox
  - 📦 Formats multiples : PNG, JPG, GIF, TIFF, etc.
  - 🎨 Filtres et transformations d'images
- Mon usage : Afficher logos dans les rapports générés

- Alternatives : OpenCV (overkill pour simple affichage), imageio

python-multipart 0.0.21

- Rôle : Parser uploads de fichiers multipart/form-data
- Avantages :
  - 📤 Requis par FastAPI pour UploadFile
  - 🔧 Streaming : Gère gros fichiers sans charger tout en RAM
  - 📦 Supporte fichiers multiples simultanés
- Mon usage : Upload CSV Catapult via /catapult/upload
- Alternatives : Aucune (dépendance obligatoire FastAPI)

---

4. FRONTEND - Framework & Build

React 19.2.0

- Rôle : Framework JavaScript pour interfaces utilisateur
- Avantages :
  - ⚛️ Composants réutilisables : Découpage modulaire de l'UI
  - 🔄 Virtual DOM : Mises à jour ultra-rapides (reconciliation)
  - 🌍 Écosystème énorme : Millions de bibliothèques disponibles
  - 💼 Standard industrie : Utilisé par Facebook, Netflix, Airbnb
  - 🎣 Hooks : useState, useEffect pour logique propre et moderne
  - 🔧 React 19 : Server Components, nouvelles optimisations
- Alternatives : Vue.js (plus simple, moins d'emplois), Angular (plus verbeux), Svelte

Vite 7.2.4

- Rôle : Build tool et serveur de développement
- Avantages :
  - ⚡ Ultra-rapide : HMR instantané (Hot Module Replacement)
  - 🚀 Dev server démarre <1s (vs 30s webpack Create React App)
  - 📦 Build optimisé : Rollup pour production (tree-shaking, code splitting)
  - 🔧 Configuration minimale : Fonctionne out-of-the-box
  - 💡 Support natif : JSX, CSS modules
  - 🎯 ESM natif en développement

- Alternatives : Create React App (obsolète, lent), Webpack (complexe), Next.js (overkill pour SPA)

ESLint 9.39.1 + Plugins

- Rôle : Linter JavaScript
- Avantages :
  - 🐛 Détecte erreurs avant exécution
  - 📏 Force conventions : Style de code uniforme
  - 🔧 Auto-fix : Corrige automatiquement beaucoup d'erreurs
  - ⚛️ Plugins React : react-hooks (vérifie règles des hooks), react-refresh (HMR)
  - 🎯 Configurable par projet
- Alternatives : Prettier seul, StandardJS

React Router DOM 7.13.0

- Rôle : Gestion du routing côté client
- Avantages :
  - 🗺️ Navigation SPA sans rechargement de page
  - 🔒 Routes protégées (ProtectedRoute)
  - 🔧 Paramètres d'URL dynamiques
  - 📦 Version 7 : Nouvelles optimisations de performance
- Alternatives : TanStack Router, Wouter (plus léger)

Axios 1.13.4

- Rôle : Client HTTP pour appels API
- Avantages :
  - 🔧 Intercepteurs : Gestion centralisée des headers JWT
  - ❌ Gestion d'erreurs : Intercepteurs pour 401/404
  - 🔄 Transformation automatique JSON
  - 📦 Annulation de requêtes
- Alternatives : Fetch natif (moins de features), SWR, React Query

---

5. FRONTEND - UI & Visualisation

Tailwind CSS 4.1.18

- Rôle : Framework CSS utility-first pour le design de l'interface
- Avantages :
  - 🎨 Design rapide : Classes utilitaires directement dans le JSX
  - 📱 Responsive : Classes breakpoints intégrées (sm, md, lg, xl)
  - 🌙 Dark mode : Support natif
  - ⚡ Purge CSS : Bundle final très léger (supprime classes non utilisées)
  - 🔧 Hautement personnalisable via tailwind.config.js
- Alternatives : Material UI (composants prêts), Bootstrap (vieillissant), CSS modules

Headless UI 2.2.9

- Rôle : Composants UI accessibles sans style imposé
- Avantages :
  - ♿ Accessibilité : ARIA intégré, navigation clavier native
  - 🎨 Liberté de style : Compatible Tailwind CSS
  - 📦 Composants : Menus, dialogs, transitions, listbox
  - ⚛️ React natif : Hooks et composants React
- Alternatives : Radix UI, shadcn/ui

Heroicons 2.2.0

- Rôle : Bibliothèque d'icônes SVG
- Avantages :
  - 🎨 Design cohérent : Icônes Tailwind UI officielles
  - ⚛️ Composants React : Import direct en JSX
  - 📦 Deux styles : outline et solid
  - ⚡ Tree-shakable : Seules les icônes utilisées sont incluses
- Alternatives : Lucide React, React Icons, FontAwesome

Recharts 3.7.0

- Rôle : Graphiques interactifs frontend
- Avantages :
  - 📊 React-friendly : Composants React natifs (pas de DOM direct)
  - 🖱️ Interactifs : Hover, click, zoom, pan
  - 🎨 Beaux par défaut : Pas besoin de config complexe

    - 📱 Responsive : S'adapte à la taille d'écran
    - 🔧 Personnalisable : Custom tooltips, axes, légendes
- Mon usage : Évolution performances joueurs, stats par match
- Alternatives : Plotly.js (3D, plus lourd), D3.js (très custom), Chart.js

---

6. INFRASTRUCTURE & Production

Docker Compose

- Rôle : Orchestration de conteneurs multi-services
- Avantages :
    - 📦 Environnement reproductible : Même config dev/prod
    - 🔧 Un fichier : docker-compose.yml définit toute l'infra
    - 🚀 Commande simple : docker-compose up lance tout
    - 💾 Volumes : Persistence PostgreSQL
    - 🌐 Networking : Isolation et communication inter-services
    - 🔄 Versions : Pin versions exactes des images
- Mon usage : PostgreSQL isolé, facile à reset, même environnement équipe
- Alternatives : Installation locale (pollution système), Kubernetes (overkill petits projets)

Python 3.12

- Rôle : Version Python du projet
- Avantages :
    - 🚀 Performances : 10-60% plus rapide que 3.10
    - 📝 Type hints améliorés
    - 🔧 Error messages : Plus clairs et précis
    - 🐛 Debugging : Meilleur traceback
- Alternatives : Python 3.11 (stable mais moins rapide)

---

7. AUTHENTIFICATION & Sécurité

JWT (JSON Web Tokens) avec OAuth2 — python-jose 3.3.0

- Rôle : Authentification et autorisation
- Avantages :

    - 🔐 Stateless : Token contient tout (pas de session serveur)
    - 👥 Multi-rôles : Coach, admin, joueur, analysiste
    - 📱 Compatible API : Mobile, SPA, services tiers
    - 🔏 Signature cryptographique : Impossible de falsifier
    - ⏱️ Expiration : Tokens courts pour sécurité
- Implémentation : FastAPI OAuth2PasswordBearer
- Alternatives : Sessions cookies (stateful, moins scalable), Auth0 (SaaS payant)

Passlib + bcrypt 4.0.1

- Rôle : Hachage sécurisé des mots de passe
- Avantages :
    - 🔒 bcrypt : Algorithme de hachage résistant aux attaques brute-force
    - 🔧 Passlib : Abstraction simple pour vérifier et créer des hashs
    - 🛡️ Protection contre les rainbow tables
    - 🔄 Salage automatique à chaque hachage
- Alternatives : argon2 (plus récent), scrypt

HTTPS via Nginx + certificats SSL

- Rôle : Chiffrement des communications
- Avantages :
    - 🔐 Chiffrement : Protège données sensibles (mots de passe, stats)
    - ✅ Certificats SSL dans le dossier certs/
    - 🔧 Configuration Nginx automatique
- Alternatives : Cloudflare (proxy), Caddy (auto HTTPS intégré)

---

8. QUALITÉ & Validation

Pydantic 2.12.5

- Rôle : Validation de données et sérialisation
- Avantages :
    - ✅ Validation automatique : Types + contraintes (min, max, regex)
    - 🔄 Sérialisation : JSON, dict, ORM
    - ⚡ Pydantic V2 : 5-50x plus rapide (Rust core)

    - 🔧 **Custom validators : Logique métier complexe**
    - 📖 **Erreurs claires : Messages détaillés**
- **Mon usage : Validation uploads CSV, requêtes API, modèles SQLModel**
- **Alternatives : Marshmallow (plus ancien), Cerberus**

Pytest 7.4.4

- **Rôle : Tests unitaires et d'intégration**
- **Avantages :**
    - 🧪 **Simple à écrire : Fonctions Python standards**
    - 🔧 **Fixtures puissantes : Setup/teardown réutilisables**
    - 📊 **Coverage : Mesure couverture de code**
    - ⚛️ **pytest-asyncio : Tests des endpoints FastAPI async**
    - 🌐 **HTTPX : Client HTTP pour tester les APIs**
- **Alternatives : unittest (stdlib, plus verbeux), nose2**